SE LAB-6

Smart Lab Equipment & Slot Reservation Portal

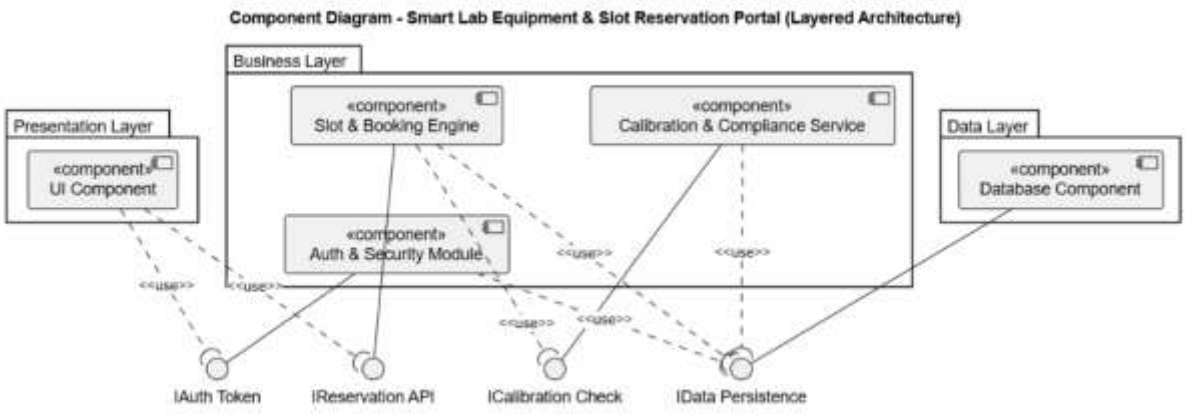
NAME: ANIKET JADHAV

SRN: PES2UG24CS063

SECTION: B

DATE: 09-09-2026

- ## Component Diagram



- ## Architectural Selection Justification

1. Architecture Selection

We chose the Layered Architecture style for the Smart Lab Equipment & Slot Reservation Portal (SLESR).

2. Concrete Scenario-Related Justifications

• Clear Separation of Operational Concerns:

The Layered Architecture cleanly isolates the Presentation Layer (student booking interface), Business Layer (slot reservation concurrency engine and calibration verification), and Data Layer (PostgreSQL/MySQL storage). This allows lab technicians and developers to

adjust equipment calibration algorithms without disrupting the booking UI or overall system layout.

• Manageable Development and Deployment Complexity:

Given the constrained scope of managing equipment slots within an academic lab setting, a Layered Architecture avoids the heavy operational and network distribution overhead associated with Microservices. It allows the core reservation logic, role-based access control, and calibration tracking to be built, tested, and deployed as a cohesive unit while keeping components modular.

3. Security Advantage

• Restricted Direct Data Access (Role-Based Access Control):

By placing the Authentication & Security Module within the Business Layer, all incoming UI calls must pass through strict application boundary validation before reaching the Data Layer. Students and Technicians cannot interface directly with the database; instead, the system enforces Role-Based Access Control (RBAC) at the business layer to restrict unauthorized administrative modifications.

4. Performance Benefit

• Low Inter-Component Latency for High Concurrency:

Layered monolithic deployment allows components (such as the Slot & Booking Engine and the Database Component) to communicate via fast in-process calls and connection pools rather than over distributed REST/gRPC network hops. This direct path is critical for acquiring database row locks within the strict 200 ms latency budget required during high-concurrency booking bursts.